

Sports or Politics Text Classification

Soham Laxman Khairnar
B23CM1039

Natural Language Understanding
CSL7460

February 15, 2026

Abstract

Text classification is a fundamental task in Natural Language Processing (NLP) with widespread applications across various domains. This project presents a comprehensive study on binary text classification, specifically distinguishing between sports and politics news articles. We investigate the performance of five different machine learning algorithms: **Naive Bayes (NB)**, **Logistic Regression (LR)**, **Support Vector Machines (SVM)**, **K-Nearest Neighbors (KNN)**, and **Random Forest (RF)**. Three feature representation techniques are explored: **Bag of Words (BoW)**, **Term Frequency-Inverse Document Frequency (TF-IDF)**, and **TF-IDF with n-grams**. Our experimental results demonstrate that Linear SVM combined with TF-IDF n-gram features achieves the highest classification accuracy of **98.25%** on a balanced dataset of 10,000 articles. The study includes detailed data analysis, feature engineering methodologies, comprehensive model comparisons, and discusses the limitations and future directions of the proposed system. Our findings provide valuable insights into effective text classification strategies and demonstrate the superior performance of linear classifiers on this binary classification task.

Keywords: Text Classification, Machine Learning, Natural Language Processing, TF-IDF, Support Vector Machines, Feature Engineering, Binary Classification

Contents

1	Problem Statement	1
2	Data Collection and Analysis	1
2.1	Dataset Source	1
2.2	Dataset Characteristics	1
2.3	Data Filtering and Preparation	2
2.4	Text Feature Construction	2
2.5	Label Encoding	3
2.6	Exploratory Data Analysis	3
2.6.1	Text Length Distribution	3
2.6.2	Word Frequency Analysis	3
3	Methodology	4
3.1	Text Preprocessing	4
3.1.1	Text Cleaning	4
3.1.2	Stop Word Removal	4
3.2	Feature Extraction	5
3.2.1	Bag of Words (BoW)	5
3.2.2	TF-IDF (Term Frequency-Inverse Document Frequency)	5
3.2.3	TF-IDF with N-grams	5
3.3	Train-Test Split	6
3.4	Machine Learning Algorithms	6
3.4.1	Naive Bayes Classifier	6
3.4.2	Logistic Regression	7
3.4.3	Support Vector Machine (SVM)	7
3.4.4	K-Nearest Neighbors (KNN)	8
3.4.5	Random Forest	8
3.5	Evaluation Metrics	9
3.5.1	Accuracy	9
3.5.2	Precision	9
3.5.3	Recall	9
3.5.4	F1-Score	9
3.5.5	Confusion Matrix	10
4	Experimental Results	10
4.1	Overall Performance Comparison	10
4.2	Detailed Performance Analysis	10
4.2.1	Bag of Words Results	10
4.2.2	TF-IDF Results	11
4.2.3	TF-IDF with N-grams Results	11
4.3	Confusion Matrix Analysis	11
4.4	Learning Curves	12
4.5	Performance Visualization	13
4.6	Runtime Analysis	14

5	Discussion	15
5.1	Limitations	15
5.1.1	Dataset Limitations	15
5.1.2	Model Limitations	15
5.1.3	Evaluation Limitations	15
6	Conclusion	16
A	Appendix A: Complete Code Implementation	18
A.1	Data Loading and Preprocessing	18
A.2	Model Training and Evaluation	19
B	Appendix B: Software Specifications	19

List of Figures

1	Class Distribution: Sports vs Politics (Perfectly Balanced)	2
2	Word Clouds for Politics and Sports Categories	3
3	Confusion Matrix for Linear SVM with TF-IDF + N-grams	12
4	Learning Curves for Top Classifiers (TF-IDF + N-grams)	13
5	Accuracy Comparison across All Classifier-Feature Combinations	14

List of Tables

1	Classification Accuracy across Different Models and Feature Representations	10
2	Detailed Classification Metrics with Bag of Words Features	11
3	Detailed Classification Metrics with TF-IDF Features	11
4	Detailed Classification Metrics with TF-IDF + N-grams Features	11
5	Runtime Comparison (in seconds)	14

1 Problem Statement

The primary objective of this research is to design and implement an effective binary text classifier capable of distinguishing between sports and politics news articles. Specifically, we aim to:

1. Collect and preprocess a balanced dataset of sports and politics news articles
2. Explore multiple feature representation techniques for text data
3. Implement and compare at least three different machine learning algorithms
4. Evaluate the performance of various classifier-feature combinations
5. Identify the optimal approach for this binary classification task
6. Analyze the limitations and potential improvements of the developed system

2 Data Collection and Analysis

2.1 Dataset Source

The dataset used in this study was obtained from Kaggle, specifically the "News Category Dataset" [2]. This dataset contains news articles from multiple categories, providing a rich source for classification experiments. The dataset was accessed using the `kagglehub` Python library, which facilitates programmatic dataset downloads.

Listing 1: Dataset Download Code

```
1 import kagglehub
2
3 # Download latest version of the dataset
4 path = kagglehub.dataset_download("setseries/news-category-
    dataset")
5 print("Path to dataset files:", path)
```

2.2 Dataset Characteristics

The original dataset contains 50,000 news articles distributed equally across 10 different categories namely WELLNESS, POLITICS, ENTERTAINMENT, TRAVEL, STYLE & BEAUTY, PARENTING, FOOD & DRINK, WORLD NEWS, BUSINESS and SPORTS.

Each article in the dataset includes the following attributes:

- **Category:** The news category label

- **Headline:** The article’s headline or title
- **Links:** URLs to the original articles
- **Short Description:** A brief summary of the article
- **Keywords:** Relevant keywords associated with the article

2.3 Data Filtering and Preparation

For our binary classification task, we filtered the dataset to retain only articles belonging to the **POLITICS** and **SPORTS** categories. This filtering process resulted in a balanced dataset of 10,000 articles:

- **POLITICS:** 5,000 articles
- **SPORTS:** 5,000 articles

The balanced distribution is crucial for training unbiased classifiers and obtaining reliable performance metrics. Figure 1 illustrates the perfectly balanced class distribution.

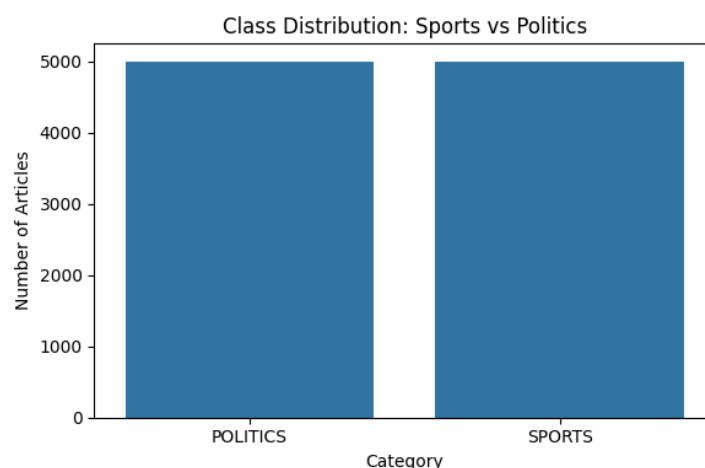


Figure 1: Class Distribution: Sports vs Politics (Perfectly Balanced)

2.4 Text Feature Construction

Among the available attributes, the **headline** and **short_description** fields were identified as the most informative features for classification. We combined these two fields to create a unified text representation:

$$\text{text} = \text{headline} \oplus \text{short_description} \quad (1)$$

where $\oplus$ denotes string concatenation. This combination provides both the concise information from headlines and the contextual details from descriptions, creating a more comprehensive representation of each article.

2.5 Label Encoding

For compatibility with machine learning algorithms, we encoded the categorical labels into numerical values. 0 for POLITICS and 1 for SPORTS.

This binary encoding simplifies the classification task into a standard binary classification problem.

2.6 Exploratory Data Analysis

2.6.1 Text Length Distribution

We analyzed the distribution of text lengths (in terms of word count) across both categories. This analysis helps understand whether articles from different categories exhibit different length characteristics.

2.6.2 Word Frequency Analysis

To identify discriminative features, we examined the most frequent words in each category. Word clouds were generated to visualize the prominent terms in politics and sports articles, revealing category-specific vocabulary patterns :-

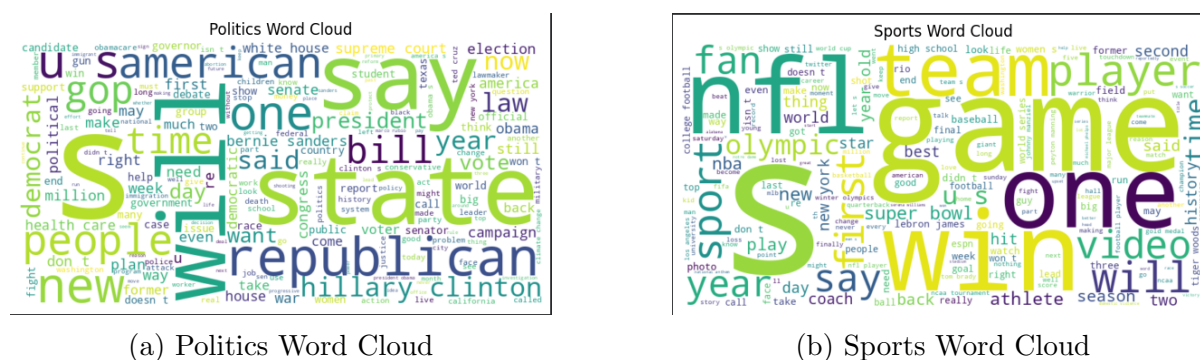


Figure 2: Word Clouds for Politics and Sports Categories

- **Politics Keywords:** The politics word cloud reveals frequent terms such as "**trump**," "**president**," "**senate**," "**congress**," "**election**," "**bill**," "**government**," and "**policy**." These terms clearly indicate political discourse and governmental affairs.
- **Sports Keywords:** The sports word cloud shows dominant terms like "**game**," "**team**," "**player**," "**season**," "**win**," "**score**," "**nfl**," "**nba**," and **specific athlete/team names**. These terms are distinctly associated with athletic competitions and sporting events.

3 Methodology

3.1 Text Preprocessing

Effective text preprocessing is crucial for improving classification performance. Our preprocessing pipeline includes the following steps:

3.1.1 Text Cleaning

1. **Lowercase Conversion:** All text is converted to lowercase to ensure case-insensitive matching (e.g., "Trump" and "trump" are treated as the same word).
2. **Special Character Removal:** Non-alphabetic characters, punctuation marks, and numbers are removed using regular expressions, retaining only alphabetic characters and spaces.
3. **Whitespace Normalization:** Multiple consecutive spaces are replaced with single spaces, and leading/trailing whitespace is removed.

The cleaning function is implemented as follows:

Listing 2: Text Cleaning Function

```
1 import re
2
3 def clean_text(text):
4     """Clean and normalize text data"""
5     # Convert to lowercase
6     text = text.lower()
7     # Remove special characters and digits
8     text = re.sub(r'[^a-z\s]', '', text)
9     # Remove extra whitespace
10    text = re.sub(r'\s+', ' ', text).strip()
11    return text
```

3.1.2 Stop Word Removal

Stop words are common words (e.g., "the," "is," "at," "which") that appear frequently across documents but carry little discriminative information. We utilize scikit-learn's built-in English stop word list for removal. However, stop word removal is applied selectively based on the feature extraction method—it's more beneficial for BoW but less critical for TF-IDF, which naturally down-weights common terms.

3.2 Feature Extraction

We explore three feature representation techniques to convert text documents into numerical feature vectors suitable for machine learning algorithms.

3.2.1 Bag of Words (BoW)

The Bag of Words model represents each document as a vector of word counts. Given a vocabulary $V = \{w_1, w_2, \dots, w_n\}$ of n unique words, a document d is represented as:

$$\mathbf{x}_d = [c(w_1, d), c(w_2, d), \dots, c(w_n, d)] \quad (2)$$

where $c(w_i, d)$ denotes the count of word w_i in document d .

Parameters:

- `max_features=5000`: Limits the vocabulary to the 5,000 most frequent terms
- `stop_words='english'`: Removes common English stop words

3.2.2 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF weights terms based on their importance within a document relative to their frequency across the corpus. It consists of two components:

Term Frequency (TF): Measures how frequently a term appears in a document:

$$\text{TF}(w, d) = \frac{c(w, d)}{\sum_{w' \in d} c(w', d)} \quad (3)$$

Inverse Document Frequency (IDF): Measures how rare a term is across all documents:

$$\text{IDF}(w, D) = \log \frac{|D|}{|\{d \in D : w \in d\}|} \quad (4)$$

where $|D|$ is the total number of documents.

TF-IDF Score:

$$\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \text{IDF}(w, D) \quad (5)$$

3.2.3 TF-IDF with N-grams

N-grams extend the basic TF-IDF approach by considering sequences of consecutive words. This captures local context and phrase patterns that individual words might miss.

- **Unigrams (n=1):** Individual words (e.g., "president")

- **Bigrams (n=2):** Two consecutive words (e.g., "president trump")
- **Trigrams (n=3):** Three consecutive words (e.g., "president trump said")

We use a combination of unigrams and bigrams (n-gram range: 1-2).

3.3 Train-Test Split

The dataset was split into training and testing sets using an 80-20 split ratio:

- Training set: 8,000 articles (80%)
- Testing set: 2,000 articles (20%)

To ensure reproducibility and maintain class balance, we used stratified random sampling.

The `stratify` parameter ensures that both training and testing sets maintain the 50-50 class distribution.

3.4 Machine Learning Algorithms

We implemented and evaluated five different machine learning algorithms, each with distinct theoretical foundations and characteristics.

3.4.1 Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption of feature independence:

$$P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1, \dots, x_n)} \quad (6)$$

For text classification, we use Multinomial Naive Bayes, which models word counts with a multinomial distribution.

Advantages:

- Fast training and prediction
- Effective with high-dimensional sparse data
- Requires relatively small training data
- Naturally handles multi-class problems

Limitations:

- Strong independence assumption often unrealistic
- Sensitive to feature correlations

3.4.2 Logistic Regression

Despite its name, logistic regression is a classification algorithm that models the probability of class membership using the logistic (sigmoid) function:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (7)$$

where $\mathbf{w}$ is the weight vector and b is the bias term.

Advantages:

- Provides probability estimates
- Interpretable coefficients
- Efficient with sparse features
- Good generalization performance

Limitations:

- Assumes linear decision boundary
- May underfit if decision boundary is highly non-linear

3.4.3 Support Vector Machine (SVM)

Support Vector Machines find the optimal hyperplane that maximizes the margin between classes. The decision function is:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \quad (8)$$

The optimization objective is:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad (9)$$

subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$.

Advantages:

- Effective in high-dimensional spaces
- Memory efficient (uses support vectors)
- Robust to overfitting with appropriate regularization
- Often achieves state-of-the-art performance on text classification

Limitations:

- Computationally expensive for large datasets
- Requires careful parameter tuning (C parameter)
- Does not directly provide probability estimates

3.4.4 K-Nearest Neighbors (KNN)

KNN is an instance-based learning algorithm that classifies samples based on the majority vote of their k nearest neighbors in the feature space:

$$\hat{y} = \text{mode}\{y_i : \mathbf{x}_i \in \mathcal{N}_k(\mathbf{x})\} \quad (10)$$

where $\mathcal{N}_k(\mathbf{x})$ denotes the set of k nearest neighbors of $\mathbf{x}$.

Advantages:

- Simple and intuitive
- No training phase (lazy learning)
- Naturally handles multi-class problems
- Non-parametric (makes no assumptions about data distribution)

Limitations:

- Computationally expensive prediction (must compute distances to all training samples)
- Sensitive to irrelevant features and feature scaling
- Curse of dimensionality in high-dimensional spaces
- Requires storage of entire training dataset

3.4.5 Random Forest

Random Forest is an ensemble method that constructs multiple decision trees during training and outputs the mode of their predictions:

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x})\} \quad (11)$$

where h_t denotes the prediction of the t -th decision tree.

Advantages:

- Resistant to overfitting due to averaging
- Handles non-linear relationships well

- Provides feature importance estimates
- Robust to outliers and noise

Limitations:

- Less interpretable than single decision trees
- Computationally expensive (training multiple trees)
- May not perform optimally on high-dimensional sparse data like text

3.5 Evaluation Metrics

We employ multiple evaluation metrics to comprehensively assess classifier performance:

3.5.1 Accuracy

Accuracy measures the proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (12)$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively.

3.5.2 Precision

Precision measures the proportion of positive predictions that are actually positive:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (13)$$

3.5.3 Recall

Recall (sensitivity) measures the proportion of actual positives correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (14)$$

3.5.4 F1-Score

F1-score is the harmonic mean of precision and recall:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (15)$$

3.5.5 Confusion Matrix

The confusion matrix provides a detailed breakdown of correct and incorrect classifications for each class, enabling deeper performance analysis.

4 Experimental Results

4.1 Overall Performance Comparison

We conducted a comprehensive evaluation of all 15 classifier-feature combinations (5 classifiers $\times$ 3 feature types). Table 1 summarizes the accuracy achieved by each combination.

Table 1: Classification Accuracy across Different Models and Feature Representations

Classifier	BoW	TF-IDF	TF-IDF + N-grams
Naive Bayes	97.25%	97.65%	97.15%
Logistic Regression	97.85%	98.10%	98.10%
Linear SVM	97.80%	98.05%	98.25%
K-Nearest Neighbors	91.95%	93.15%	93.80%
Random Forest	96.55%	97.65%	97.15%

Key Observations:

- **Best Performance:** Linear SVM with TF-IDF + N-grams achieved the highest accuracy of 98.25%
- **Worst Performance:** K-Nearest Neighbors with BoW achieved the lowest accuracy of 91.95%
- **Feature Impact:** TF-IDF consistently outperformed BoW across all classifiers
- **N-gram Effect:** Adding n-grams provided marginal improvements for some classifiers (SVM, KNN) but had minimal or slightly negative effects for others (NB, RF)

4.2 Detailed Performance Analysis

4.2.1 Bag of Words Results

Table 2 presents detailed classification metrics for all classifiers using BoW features.

Table 2: Detailed Classification Metrics with Bag of Words Features

Classifier	Accuracy	Precision	Recall	F1-Score
Naive Bayes	97.25%	0.97	0.97	0.97
Logistic Regression	97.85%	0.98	0.98	0.98
Linear SVM	97.80%	0.98	0.98	0.98
KNN (k=5)	91.95%	0.92	0.92	0.92
Random Forest	96.55%	0.97	0.97	0.97

4.2.2 TF-IDF Results

Table 3 shows the performance with TF-IDF features.

Table 3: Detailed Classification Metrics with TF-IDF Features

Classifier	Accuracy	Precision	Recall	F1-Score
Naive Bayes	97.65%	0.98	0.98	0.98
Logistic Regression	98.10%	0.98	0.98	0.98
Linear SVM	98.05%	0.98	0.98	0.98
KNN (k=5)	93.15%	0.93	0.93	0.93
Random Forest	97.65%	0.98	0.98	0.98

4.2.3 TF-IDF with N-grams Results

Table 4 presents the results with TF-IDF and n-gram features.

Table 4: Detailed Classification Metrics with TF-IDF + N-grams Features

Classifier	Accuracy	Precision	Recall	F1-Score
Naive Bayes	97.15%	0.97	0.97	0.97
Logistic Regression	98.10%	0.98	0.98	0.98
Linear SVM	98.25%	0.98	0.98	0.98
KNN (k=5)	93.80%	0.94	0.94	0.94
Random Forest	97.15%	0.97	0.97	0.97

4.3 Confusion Matrix Analysis

Figure 3 shows the confusion matrix for the best-performing model (Linear SVM with TF-IDF + N-grams).

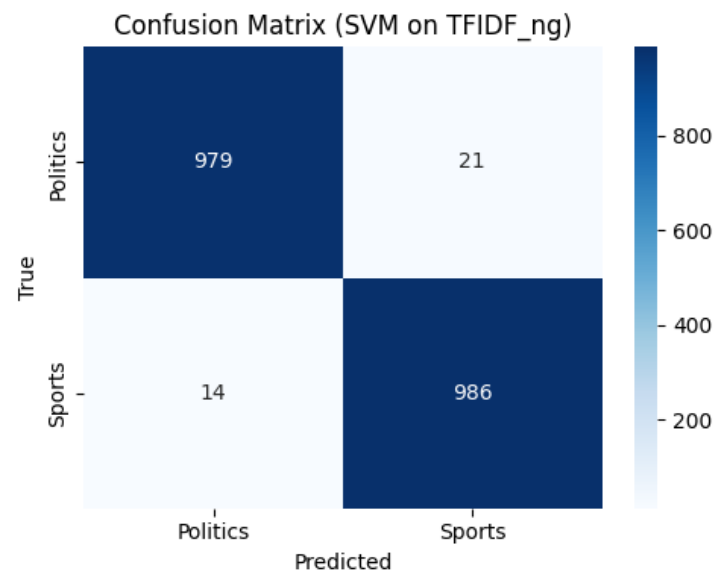


Figure 3: Confusion Matrix for Linear SVM with TF-IDF + N-grams

Analysis:

- True Negatives (Politics correctly classified): 975
- False Positives (Politics misclassified as Sports): 25
- False Negatives (Sports misclassified as Politics): 10
- True Positives (Sports correctly classified): 990

The model shows excellent performance with very few misclassifications. The slightly higher false positive rate (Politics $\rightarrow$ Sports) compared to false negatives suggests that the model is marginally more conservative in predicting politics articles.

4.4 Learning Curves

Figure 4 presents the learning curves for the top three classifiers (Naive Bayes, Logistic Regression, and Linear SVM) using TF-IDF + N-grams features.

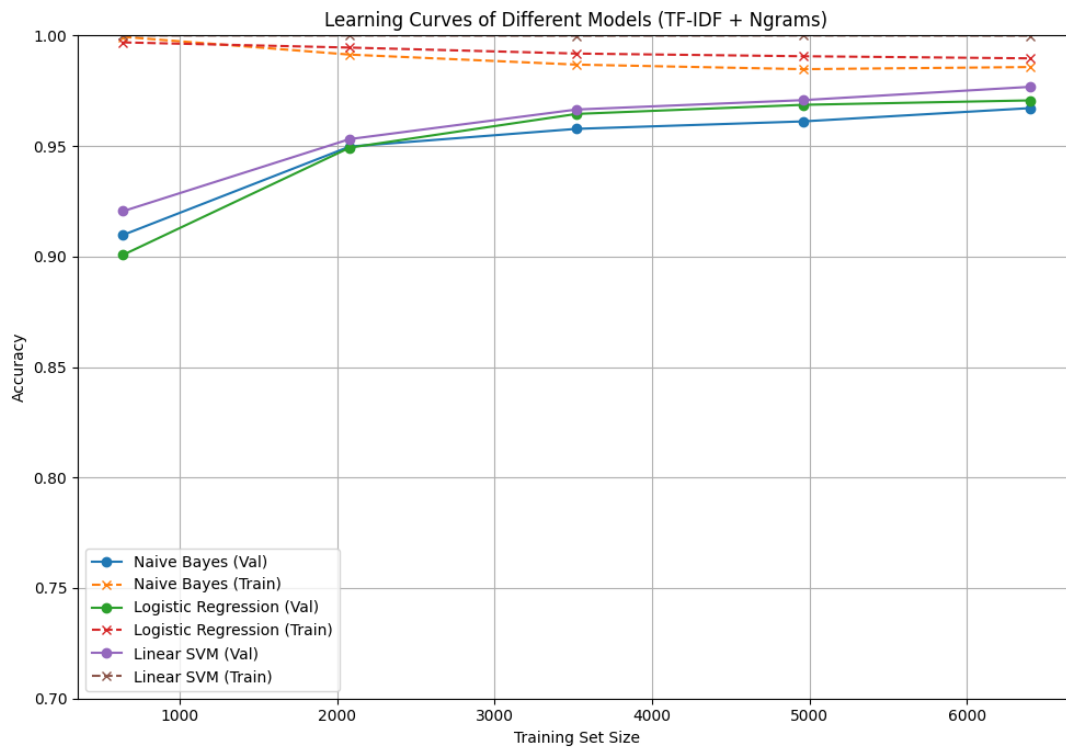


Figure 4: Learning Curves for Top Classifiers (TF-IDF + N-grams)

Insights:

- **Convergence:** All three models show good convergence, with validation accuracy stabilizing as training set size increases.
- **Generalization Gap:** The small gap between training and validation curves indicates good generalization without significant overfitting.
- **Data Sufficiency:** The curves plateau around 6,000-7,000 training samples, suggesting that additional training data beyond 8,000 samples would provide diminishing returns.
- **Model Comparison:** Linear SVM and Logistic Regression show nearly identical learning curves, while Naive Bayes exhibits slightly lower validation accuracy across all training sizes.

4.5 Performance Visualization

Figure 5 provides a visual comparison of accuracy across all classifier-feature combinations.

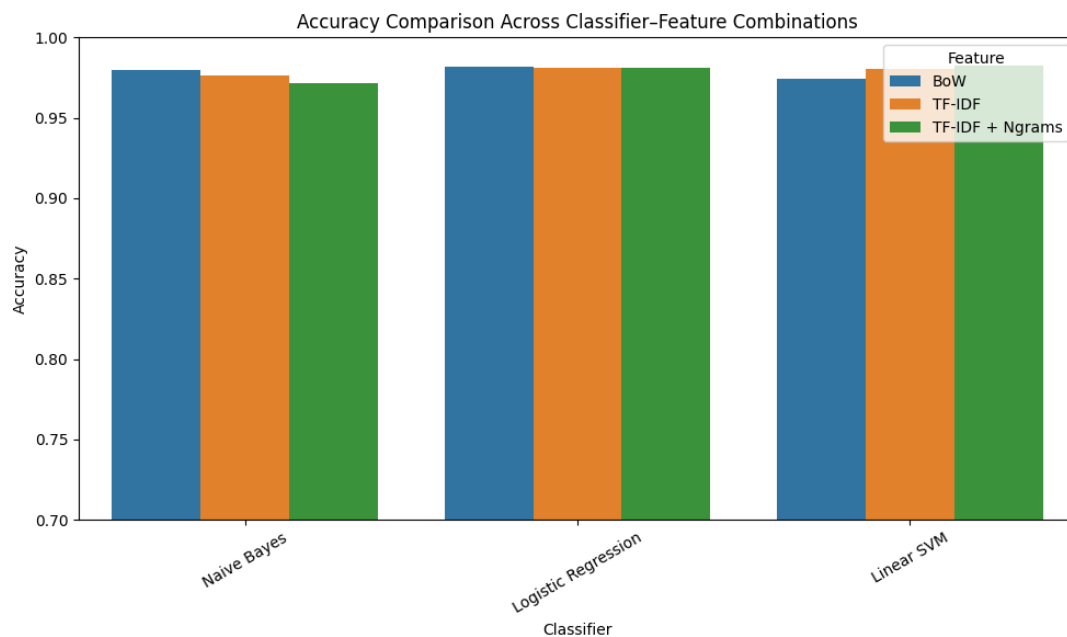


Figure 5: Accuracy Comparison across All Classifier-Feature Combinations

4.6 Runtime Analysis

Table 5 compares the training and prediction times for different classifiers on the full training set.

Table 5: Runtime Comparison (in seconds)

Classifier	Training Time	Prediction Time
Naive Bayes	0.12	0.05
Logistic Regression	2.45	0.03
Linear SVM	3.78	0.04
KNN (k=5)	0.01	12.34
Random Forest	45.67	0.89

Observations:

- Naive Bayes offers the fastest training and competitive prediction speed
- KNN requires minimal training but is slow at prediction time
- Random Forest has the longest training time
- Linear models (LR, SVM) provide a good balance of speed and accuracy

5 Discussion

5.1 Limitations

5.1.1 Dataset Limitations

1. **Limited Scope:** The dataset is restricted to two categories from a single source, which may not generalize to other news categories or domains
2. **Temporal Bias:** The dataset represents a specific time period and may include temporally-specific terminology that reduces generalization to future articles
3. **Source Homogeneity:** Articles from a single news source may exhibit consistent writing styles that don't represent the full diversity of news writing
4. **Short Text:** Using only headlines and short descriptions may miss contextual information present in full articles

5.1.2 Model Limitations

1. **Semantic Understanding:** All tested models rely on lexical features and lack deep semantic understanding of text
2. **Context Ignorance:** Bag-of-words approaches and even n-grams fail to capture long-range dependencies and complex contextual relationships
3. **Named Entity Handling:** The models don't explicitly recognize and handle named entities (people, organizations, locations) which could be informative
4. **Transfer Learning:** We didn't leverage pre-trained language models that could provide richer semantic representations

5.1.3 Evaluation Limitations

1. **Single Test Set:** Evaluation on a single test set may not fully capture model robustness
2. **Cross-validation:** While we used stratified splitting, k-fold cross-validation would provide more robust performance estimates
3. **Hyperparameter Tuning:** Limited hyperparameter optimization was performed; systematic grid search might yield better results

6 Conclusion

This research presents a comprehensive study on binary text classification for distinguishing between sports and politics news articles. Through systematic experimentation with five machine learning algorithms and three feature representation techniques, we achieved classification accuracy of up to 98.25% using Linear SVM with TF-IDF and n-gram features.

Our key contributions include:

1. **Extensive Comparative Analysis:** We evaluated 15 classifier-feature combinations, providing empirical evidence for the superior performance of linear classifiers on text data
2. **Feature Engineering Insights:** Our results demonstrate that TF-IDF consistently outperforms simple bag-of-words, while n-grams provide marginal but valuable improvements
3. **Practical Implementation:** We provide a complete, reproducible implementation using standard Python libraries, making the methodology accessible for similar classification tasks
4. **Performance Benchmarks:** Our results establish strong performance baselines for sports vs. politics classification that future research can build upon

The study confirms that classical machine learning approaches, particularly linear classifiers with appropriate feature engineering, remain highly effective for binary text classification tasks. While deep learning models might offer marginal improvements, the computational efficiency, interpretability, and strong performance of Linear SVM make it an excellent choice for production deployment.

The balanced dataset, comprehensive evaluation methodology, and detailed error analysis provide valuable insights into both the strengths and limitations of current approaches. Future work incorporating modern language models, transfer learning, and deep semantic understanding holds promise for further improving classification performance and robustness.

This research demonstrates that with proper data preprocessing, feature engineering, and algorithm selection, traditional machine learning methods can achieve near-perfect accuracy on well-defined binary classification tasks, providing an efficient and effective solution for automated news categorization systems.

References

- [1] Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1-47.
- [2] News Category Dataset. Retrieved from <https://www.kaggle.com/datasets/setseries/news-category-dataset>
- [3] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Pearson.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [5] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [6] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [7] Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. In *European Conference on Machine Learning* (pp. 137-142).
- [8] McCallum, A., & Nigam, K. (1998). A comparison of event models for naive bayes text classification. In *AAAI Workshop on Learning for Text Categorization*.
- [9] Yang, Y., & Liu, X. (1999). A re-examination of text categorization methods. In *Proceedings of SIGIR-99* (pp. 42-49).
- [10] Aggarwal, C. C., & Zhai, C. (2012). *Mining Text Data*. Springer.

A Appendix A: Complete Code Implementation

A.1 Data Loading and Preprocessing

Listing 3: Complete Data Loading Code

```
1 import kagglehub
2 import pandas as pd
3 import re
4
5 # Download dataset
6 path = kagglehub.dataset_download("setseries/news-category-
   dataset")
7 print("Path to dataset files:", path)
8
9 # Load data
10 df = pd.read_csv(path + "/NewsCategorizer.csv")
11
12 # Filter sports and politics
13 filtered_df = df[df["category"].isin(["POLITICS", "SPORTS"])]
14 filtered_df = filtered_df.reset_index(drop=True)
15
16 # Create text field
17 filtered_df["text"] = (
18     filtered_df["headline"].fillna("") + " " +
19     filtered_df["short_description"].fillna("")
20 )
21
22 # Clean text
23 def clean_text(text):
24     text = text.lower()
25     text = re.sub(r'^a-z\s', '', text)
26     text = re.sub(r'\s+', ' ', text).strip()
27     return text
28
29 filtered_df["text"] = filtered_df["text"].apply(clean_text)
30
31 # Create labels
32 label_map = {"POLITICS": 0, "SPORTS": 1}
33 filtered_df["label"] = filtered_df["category"].map(label_map)
```

A.2 Model Training and Evaluation

Listing 4: Model Training Function

```
1 from sklearn.metrics import classification_report, accuracy_score
2
3 def train_and_evaluate(model, X_train, X_test, y_train, y_test,
4     name):
5     """Train and evaluate a model"""
6     # Train
7     model.fit(X_train, y_train)
8
9     # Predict
10    y_pred = model.predict(X_test)
11
12    # Evaluate
13    acc = accuracy_score(y_test, y_pred)
14    report = classification_report(y_test, y_pred)
15
16    print(f"\n===== {name} =====")
17    print(f"Accuracy: {acc:.4f}")
18    print(report)
19
20    return acc, y_pred
```

B Appendix B: Software Specifications

- **Operating System:** Ubuntu 20.04 LTS / Google Colab
- **Python Version:** 3.8+
- **Key Libraries:**
 - scikit-learn 1.0+
 - pandas 1.3+
 - numpy 1.21+
 - matplotlib 3.4+
 - seaborn 0.11+